30

条件变量

到目前为止，我们介绍了锁（lock）的概念，也看到了怎样把硬件和操作系统的支持恰当地结合起来，正确地实现一把锁。遗憾的是，要编写并发程序（concurrent program），光有锁这一种原语还不够。

具体来说，很多时候线程（thread）需要先检查某个**条件**（condition）是否成立，然后才能继续执行。例如，父线程（parent thread）可能想先确认子线程（child thread）已经执行完毕，再继续往下走（这通常叫作 `join()`；这样的等待该怎么实现呢？我们来看图 30.1。

```
1  void *child(void *arg) {
2      printf("child\n");
3      // XXX 怎么表示我们已经完成了?
4      return NULL;
5  }
6
7  int main(int argc, char *argv[]) {
8      printf("parent: begin\n");
9      pthread_t c;
10     Pthread_create(&c, NULL, child, NULL); // 子线程
11     // XXX 怎么等待子线程?
12     printf("parent: end\n");
13     return 0;
14 }
```

图 30.1：**父线程等待子线程**

我们希望看到的输出是这样的：

```
parent: begin
child
parent: end
```

我们可以试试用一个共享变量，如图 30.2 所示。这个办法一般能行，但效率极低：父线程会一直自旋（spin），白白浪费 CPU 时间。我们真正想要的，是某种能让父线程睡眠的办法，等到它所等的条件（例如子线程执行完毕）成立时再醒来。

```
1  volatile int done = 0;
2
3  void *child(void *arg) {
4      printf("child\n");
5      done = 1;
6      return NULL;
7  }
8
9  int main(int argc, char *argv[]) {
10     printf("parent: begin\n");
11     pthread_t c;
12     Pthread_create(&c, NULL, child, NULL); // 子线程
13     while (done == 0)
14         ; // 自旋
15     printf("parent: end\n");
16     return 0;
17 }
```

图 30.2：**父线程等待子线程：基于自旋的方法**

关键问题：如何等待一个条件

在多线程（multi-threaded）程序中，线程常常需要等某个条件成立之后再继续执行。最简单的做法是一直自旋，直到条件成立，但这样做效率极低，会浪费 CPU 周期，在某些情况下甚至是错误的。那么，线程应该怎样等待一个条件呢?

30.1 定义和例程

要等待某个条件成立，线程可以使用所谓的**条件变量**（condition variable）。**条件变量**（condition variable）是一个显式的队列（queue）：当程序执行时的某种状态（也就是某个**条件**）不符合要求时，线程可以把自己放进这个队列（即在该条件上**等待**，waiting）；另一个线程改变了这个状态之后，就可以唤醒一个（或多个）等待中的线程，让它们继续执行（即在该条件上**发信号**，signaling）。这个思想可以追溯到 Dijkstra 使用的“私用信号量”（private semaphores）[D68]；后来，Hoare 在研究管程（monitor）时，把一个类似的思想命名为“条件变量” [H74]。

声明条件变量很简单，写一句 `pthread_cond_t c;`，就把 `c` 声明成了条件变量（注意：还需要正确地初始化）。条件变量有两个相关的操作：`wait()` 和 `signal()`。线程想让自己睡眠时，调用 `wait()`；线程修改了程序中的某些东西，想唤醒在这个条件上睡眠等待的线程时，调用 `signal()`。具体来说，POSIX 调用是这样的：

```
pthread_cond_wait(pthread_cond_t *c, pthread_mutex_t *m);
pthread_cond_signal(pthread_cond_t *c);
```

为简单起见，我们常把它们称为 `wait()` 和 `signal()`。你可能注意到，`wait()` 调用还要传入一个互斥锁（mutex）作为参数；它假定调用 `wait()` 时这个互斥锁已经

 © 2008–25, ARPACI-DUSSEAU

```
1  int done = 0;
2  pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
3  pthread_cond_t c = PTHREAD_COND_INITIALIZER;
4
5  void thr_exit() {
6      Pthread_mutex_lock(&m);
7      done = 1;
8      Pthread_cond_signal(&c);
9      Pthread_mutex_unlock(&m);
10 }
11
12 void *child(void *arg) {
13     printf("child\n");
14     thr_exit();
15     return NULL;
16 }
17
18 void thr_join() {
19     Pthread_mutex_lock(&m);
20     while (done == 0)
21         Pthread_cond_wait(&c, &m);
22     Pthread_mutex_unlock(&m);
23 }
24
25 int main(int argc, char *argv[]) {
26     printf("parent: begin\n");
27     pthread_t p;
28     Pthread_create(&p, NULL, child, NULL);
29     thr_join();
30     printf("parent: end\n");
31     return 0;
32 }
```

图 30.3：**父线程等待子线程：使用条件变量**

加锁（locked）。`wait()` 的职责是释放这把锁，并让调用它的线程睡眠，这两步要原子地（atomically）完成；线程被唤醒以后（在别的线程给它发了信号（signal）之后），必须重新获取这把锁，才能返回到调用者。之所以这么复杂，是为了防止线程在让自己睡眠的过程中出现某些竞态条件（race condition，也叫竞争条件）。为了更好地理解这一点，我们来看看 join 问题的解法（图 30.3）。

需要考虑两种情况。第一种情况是，父线程创建了子线程，但自己继续运行（假设只有一个处理器），于是立即调用 `thr_join()`，等待子线程完成。这时，父线程会获取锁，检查子线程是否已经完成（还没有），然后调用 `wait()` 让自己睡眠（同时释放锁）。子线程最终会运行，打印出消息“child”，然后调用 `thr_exit()` 唤醒父线程；这段代码只是获取锁，设置状态变量（state variable）`done`，然后给父线程发信号，把它唤醒。最后，父线程运行（从 `wait()` 返回时已经持有锁），释放锁，并打印出最后一条消息“parent: end”。

```
1  void thr_exit() {
2      Pthread_mutex_lock(&m);
3      Pthread_cond_signal(&c);
4      Pthread_mutex_unlock(&m);
5  }
6
7  void thr_join() {
8      Pthread_mutex_lock(&m);
9      Pthread_cond_wait(&c, &m);
10     Pthread_mutex_unlock(&m);
11 }
```

图 30.4：**父线程等待：没有状态变量**

第二种情况是，子线程一被创建就立即运行，把 done 设为 1，调用 signal 唤醒睡眠的线程（但并没有线程在睡眠，所以直接返回），然后结束。之后父线程运行，调用 thr_join()，发现 done 是 1，于是不用等待，直接返回。

最后说一点：你可能注意到，父线程在决定要不要在条件上等待时，用的是 while 循环，而不只是一条 if 语句。按照程序的逻辑，这样做似乎并非绝对必要，但它总是一个好习惯，后面我们会看到原因。

为了确保你理解 thr_exit() 和 thr_join() 代码中每一部分的重要性，我们来试几种别的实现。首先，你可能想知道是否真的需要状态变量（state variable）done。如果代码写成下面这样会怎样？（图 30.4）

遗憾的是，这种做法是错的。设想子线程被创建后立即运行，并且马上调用了 thr_exit()；这时子线程会发信号，但并没有线程在这个条件上睡眠。等父线程运行时，它会直接调用 wait，然后就卡在那里，再也不会有线程来唤醒它。从这个例子你应该能体会到状态变量（state variable）done 的重要性：它记录了各个线程关心的那个值。睡眠、唤醒和加锁（locking），都是围绕它展开的。

这里（图 30.5）是另一个糟糕的实现。在这个例子中，我们设想发信号和等待时都不需要持有锁。这里会出什么问题？想一想[1]！

```
1  void thr_exit() {
2      done = 1;
3      Pthread_cond_signal(&c);
4  }
5
6  void thr_join() {
7      if (done == 0)
8          Pthread_cond_wait(&c);
9  }
```

图 30.5：**父线程等待：没有锁**

[1]注意，这个例子并不是“真实”的代码，因为调用 pthread_cond_wait() 时，除了条件变量，总是还需要传入一个互斥锁；这里只是为了举这个反面例子，假装接口不需要互斥锁。

提示：发信号时总是持有锁

虽然并非所有情况都严格需要，但使用条件变量时，发信号时持有锁可能是最简单、也最好的做法。上面的例子展示了一种为了正确性必须持有锁的情况；不过也有一些情况，不持有锁大概也没问题，但多半还是应该避免。所以，为简单起见，**调用 signal 时要持有锁**。

这条提示反过来，也就是调用 wait 时要持有锁，就不只是一条提示了，而是 wait 的语义所规定的。因为 wait 总是：（a）假定你调用它时已经持有锁；（b）在让调用者睡眠时释放这把锁；（c）在返回之前重新获取这把锁。因此，把这条提示推广一下也是对的：**调用 signal 或 wait 时都持有锁**，你就总不会出问题。

这里的问题是一个微妙的竞态条件。具体来说，如果父线程调用 `thr_join()`，然后检查 `done` 的值，会发现它是 0，于是准备去睡眠。但就在它调用 wait 去睡眠之前，父线程被中断（interrupted），子线程开始运行。子线程把状态变量（state variable）`done` 改成 1，然后发信号，可此时没有线程在等待，所以没有线程被唤醒。等父线程再次运行时，它就会永远睡下去，真让人难过。

希望你能从这个简单的 join 例子中，看出正确使用条件变量的一些基本要求。为了确保你真正理解，我们接下来看一个更复杂的例子：**生产者/消费者**（producer/consumer）问题，也叫**有界缓冲区**（bounded buffer）问题。

30.2 生产者/消费者（有界缓冲区）问题

本章要面对的下一个同步问题叫作**生产者/消费者**（producer/consumer）问题，有时也叫**有界缓冲区**（bounded buffer）问题，最早由 Dijkstra 提出 [D72]。事实上，正是这个生产者/消费者问题，促使 Dijkstra 和他的同事发明了通用的信号量（semaphore），它既可以当锁用，也可以当条件变量用 [D01]；后面我们还会进一步学习信号量。

设想有一个或多个生产者线程（producer thread），以及一个或多个消费者线程（consumer thread）。生产者生成数据项，把它们放进缓冲区（buffer）；消费者从缓冲区里取出这些数据项，以某种方式消费掉。

这种安排在许多实际系统中都会出现。例如，在多线程的 Web 服务器中，生产者把 HTTP 请求放进一个工作队列（work queue），也就是有界缓冲区（bounded buffer）；消费者线程（consumer thread）从这个队列中取出请求并处理。

把一个程序的输出用管道（pipe）传给另一个程序时，也会用到有界缓冲区（bounded buffer），例如 `grep foo file.txt | wc -l`。这个例子并发（concurrently）运行两个进程（process）：`grep` 把 `file.txt` 中含有字符串 `foo` 的行写到它以为的标准输出（standard output）上；UNIX shell 把这个输出重定向（redirect）到所谓的 UNIX 管道。管道的另一端连着进程 `wc` 的标准输入（standard input），`wc` 只是统计输入流中有多少行，然后打印出结果。因此，`grep` 进程是生产者，`wc` 进程是消费者，它们之间是一个内核中的有界缓冲区（bounded buffer）；而在这个例子中，你只是那个开心的用户。

 © 2008–25, ARPACI-DUSSEAU

```
1  int buffer;
2  int count = 0; // 初始为空
3
4  void put(int value) {
5      assert(count == 0);
6      count = 1;
7      buffer = value;
8  }
9
10 int get() {
11     assert(count == 1);
12     count = 0;
13     return buffer;
14 }
```

图 30.6：**put 和 get 例程（第 1 版）**

由于有界缓冲区（bounded buffer）是共享资源，访问它时当然必须同步，以免[2]出现竞态条件。为了更好地理解这个问题，我们来看一些实际的代码。

我们首先需要一个共享缓冲区，生产者把数据放进去，消费者从里面取数据。为简单起见，我们只用一个整数（你当然可以想象往这个位置放一个指向某个数据结构（data structure）的指针），再加上两个内部例程：一个把值放进共享缓冲区，一个从缓冲区取出值。详见图 30.6（第 6 页）。

很简单，对吧？`put()` 例程假定缓冲区是空的（并用断言检查这一点），然后只是把一个值放进共享缓冲区，再把 `count` 设为 1，标记缓冲区已满。`get()` 例程正好相反：它把缓冲区置空（即把 `count` 设为 0），并返回那个值。不用担心这个共享缓冲区只有一个位置；稍后我们会把它推广成一个能容纳多项的队列，那会比听起来还要有趣。

现在，我们需要写一些例程，让它们知道什么时候可以访问缓冲区，往里放数据或从中取数据。条件应该很明显：只有在 `count` 为 0（即缓冲区为空）时，才往缓冲区放数据；只有在 `count` 为 1（即缓冲区已满）时，才从缓冲区取数据。如果我们写出的同步代码让生产者往满的缓冲区里放数据，或者让消费者从空的缓冲区里取数据，那就是做错了（在这段代码中，断言会被触发）。

这项工作由两类线程完成，一类我们称为**生产者**线程，另一类称为**消费者**线程。图 30.7 给出了它们的代码：生产者把一个整数放进共享缓冲区，一共放 `loops` 次；消费者则（永不停止地）从共享缓冲区取出数据，每取一次，就把取到的数据项打印出来。

有问题的方案

现在假设只有一个生产者和一个消费者。显然，`put()` 和 `get()` 例程中都有临界区（critical section），因为 `put()` 会更新缓冲区，`get()` 会读取缓冲区。然而，只是在代码外面加一把锁是不行的，我们还需要别的东西。毫不意外，这个“别的东西”就是一些条件

[2]我们在这里用了一个相当古老的英语词 lest（以免），连虚拟语气都用上了。

```
1  void *producer(void *arg) {
2      int i;
3      int loops = (int) arg;
4      for (i = 0; i < loops; i++) {
5          put(i);
6      }
7  }
8
9  void *consumer(void *arg) {
10     while (1) {
11         int tmp = get();
12         printf("%d\n", tmp);
13     }
14 }
```

图 30.7：**生产者/消费者线程（第 1 版）**

变量。在这次（有问题的）初步尝试中（图 30.8），我们使用了一个条件变量 `cond`，以及与它关联的锁 `mutex`。

我们来看一下生产者和消费者之间的发信号逻辑。生产者想要填充缓冲区时，会等待缓冲区变空（p1–p3）。消费者的逻辑完全一样，只是等待的条件不同：缓冲区变满（c1–c3）。

只有一个生产者和一个消费者时，图 30.8 中的代码能够正常工作。然而，如果这类线程不止一个（例如有两个消费者），这个方案就有两个严重的问题。是哪两个呢?

……（在这里停下来想一想）……

我们先来理解第一个问题，它和等待之前的那条 `if` 语句有关。假设有两个消费者（T_{c1} 和 T_{c2}）和一个生产者（T_p）。首先，一个消费者（T_{c1}）运行；它获取锁（c1），检查有没有可以消费的缓冲区（c2），发现没有，于是等待（c3），这会释放锁。

接着，生产者（T_p）运行。它获取锁（p1），检查是否所有缓冲区都满了（p2），发现并没有，就接着往缓冲区里填数据（p4）。然后，生产者发信号，表示有一个缓冲区已经填满（p5）。关键在于，这会让第一个消费者（T_{c1}）不再在条件变量上睡眠，而是进入就绪队列（ready queue）；T_{c1} 现在可以运行了（只是还没有运行）。之后，生产者继续执行，直到发现缓冲区已满，于是去睡眠（p6、p1–p3）。

问题就出在这里：另一个消费者（T_{c2}）趁机溜了进来，把缓冲区里仅有的那个值消费掉了（c1、c2、c4、c5、c6；因为缓冲区是满的，所以跳过了 c3 处的等待）。现在假设 T_{c1} 运行；在从 wait 返回之前，它重新获取锁，然后返回。接着它调用 `get()`（c4），可缓冲区里已经没有可以消费的数据了！断言被触发，代码没有按预期工作。显然，我们本该设法阻止 T_{c1} 去消费，因为 T_{c2} 已经溜进来，把缓冲区里刚生产出来的那个值消费掉了。图 30.9 展示了每个线程执行的动作，以及它的调度状态随时间的变化；调度状态分为就绪（ready）、运行（running）和睡眠（sleeping）三种。

问题的原因很简单：生产者唤醒 T_{c1} 之后、T_{c1} 真正运行之前，有界缓冲区（bounded buffer）的状态（因为 T_{c2}）发生了变化。给一个线程发信号只是把它唤醒；因此，这只是一个提示，表示外界的状态变了（在这个例子中，是缓冲区里放进了一个值），但并不能保证被唤

```
1  int loops; // 必须在某处初始化……
2  cond_t  cond;
3  mutex_t mutex;
4
5  void *producer(void *arg) {
6      int i;
7      for (i = 0; i < loops; i++) {
8          Pthread_mutex_lock(&mutex);            // p1
9          if (count == 1)                        // p2
10             Pthread_cond_wait(&cond, &mutex); // p3
11         put(i);                                // p4
12         Pthread_cond_signal(&cond);            // p5
13         Pthread_mutex_unlock(&mutex);          // p6
14     }
15 }
16
17 void *consumer(void *arg) {
18     int i;
19     for (i = 0; i < loops; i++) {
20         Pthread_mutex_lock(&mutex);            // c1
21         if (count == 0)                        // c2
22             Pthread_cond_wait(&cond, &mutex); // c3
23         int tmp = get();                       // c4
24         Pthread_cond_signal(&cond);            // c5
25         Pthread_mutex_unlock(&mutex);          // c6
26         printf("%d\n", tmp);
27     }
28 }
```

图 30.8：**生产者/消费者：一个条件变量加 if 语句**

醒的线程运行时，状态仍然符合要求。对信号含义的这种理解，通常称为 **Mesa 语义**（Mesa semantics），得名于最早以这种方式实现条件变量的研究 [LR80]；与之相对的叫作 **Hoare 语义**（Hoare semantics），它实现起来更难，但提供的保证更强：被唤醒的线程会在被唤醒后立即运行 [H74]。几乎所有实际做出来的系统，用的都是 Mesa 语义（Mesa semantics）。

好一些，但仍有问题：用 while，不用 if

幸运的是，修起来很容易（图 30.10）：把 `if` 改成 `while`。想一想为什么这样就行了：现在，消费者 T_{c1} 醒来后，会（在持有锁的情况下）立即重新检查共享变量的状态（c2）。如果这时缓冲区是空的，消费者就回去接着睡（c3）。生产者中相应的 `if` 也改成了 `while`（p2）。

由于采用的是 Mesa 语义（Mesa semantics），使用条件变量时要记住一条简单的规则：**总是使用 while 循环**。有时候你并不需要重新检查条件，但重新检查总是安全的；照做就是了，你会很开心的。

 © 2008–25, ARPACI-DUSSEAU

T_{c1}	状态	T_{c2}	状态	T_p	状态	**计数**	说明
c1	运行		就绪		就绪	0	
c2	运行		就绪		就绪	0	
c3	睡眠		就绪		就绪	0	没有数据可取
	睡眠		就绪	p1	运行	0	
	睡眠		就绪	p2	运行	0	
	睡眠		就绪	p4	运行	1	缓冲区已满
	就绪		就绪	p5	运行	1	T_{c1} 被唤醒
	就绪		就绪	p6	运行	1	
	就绪		就绪	p1	运行	1	
	就绪		就绪	p2	运行	1	
	就绪		就绪	p3	睡眠	1	缓冲区已满，睡眠
	就绪	c1	运行		睡眠	1	T_{c2} 溜了进来……
	就绪	c2	运行		睡眠	1	
	就绪	c4	运行		睡眠	0	……并取走了数据
	就绪	c5	运行		就绪	0	T_p 被唤醒
	就绪	c6	运行		就绪	0	
c4	运行		就绪		就绪	0	糟了！没有数据

图 30.9：**线程轨迹：有问题的方案（第 1 版）**

然而，这段代码仍然有一个 bug，也就是前面提到的两个问题中的第二个。你能看出来吗？它和只有一个条件变量这一点有关。在往下读之前，先试着想出问题在哪里。一定要想！（停下来想一想，或者闭上眼睛……）

我们来确认一下你是不是想对了，或者说，确认一下你现在还醒着，正在读书里的这一部分。问题出现在两个消费者先运行（T_{c1} 和 T_{c2}）并且都去睡眠（c3）的时候。接着生产者运行，往缓冲区里放了一个值，并唤醒其中一个消费者（比如 T_{c1}）。然后，生产者回到循环开头（中途释放锁、又重新获取锁），想往缓冲区里放更多数据；由于缓冲区已满，生产者只好在条件上等待（于是睡眠）。现在，一个消费者（T_{c1}）可以运行，另外两个线程 T_{c2} 和 T_p 在条件上睡眠。麻烦马上就要来了：事情变得刺激起来了！

接着，消费者 T_{c1} 从 `wait()` 返回（c3），醒了过来，重新检查条件（c2），发现缓冲区是满的，就把这个值消费掉（c4）。关键在于，这个消费者随后会在条件上发信号（c5），而这只会唤醒一个正在睡眠的线程。那么，它应该唤醒哪个线程呢？

由于消费者刚把缓冲区清空，它显然应该唤醒生产者。然而，如果它唤醒的是消费者 T_{c2}（这完全有可能，取决于等待队列是怎么管理的），就出问题了。具体来说，消费者 T_{c2} 醒来后会发现缓冲区是空的（c2），于是又回去睡眠（c3）。而有值要放进缓冲区的生产者 T_p 仍在睡眠。另一个消费者线程（consumer thread）T_{c1} 也回去睡了。三个线程全都在睡眠，这显然是个 bug；图 30.11 把这场可怕灾难的残酷经过一步步列了出来。

发信号显然是必要的，但必须更有针对性。消费者不应该唤醒其他消费者，只应该唤醒生产者，反之亦然。

```
1  int loops; // 必须在某处初始化……
2  cond_t  cond;
3  mutex_t mutex;
4
5  void *producer(void *arg) {
6      int i;
7      for (i = 0; i < loops; i++) {
8          Pthread_mutex_lock(&mutex);            // p1
9          while (count == 1)                     // p2
10             Pthread_cond_wait(&cond, &mutex); // p3
11         put(i);                                // p4
12         Pthread_cond_signal(&cond);            // p5
13         Pthread_mutex_unlock(&mutex);          // p6
14     }
15 }
16
17 void *consumer(void *arg) {
18     int i;
19     for (i = 0; i < loops; i++) {
20         Pthread_mutex_lock(&mutex);            // c1
21         while (count == 0)                     // c2
22             Pthread_cond_wait(&cond, &mutex); // c3
23         int tmp = get();                       // c4
24         Pthread_cond_signal(&cond);            // c5
25         Pthread_mutex_unlock(&mutex);          // c6
26         printf("%d\n", tmp);
27     }
28 }
```

图 30.10：**生产者/消费者：一个条件变量加 while**

单缓冲区的生产者/消费者方案

这里的解决办法同样很简单：用两个条件变量，而不是一个，这样在系统状态变化时，就能正确地通知应该醒来的那一类线程。图 30.12 给出了修改后的代码。

在这段代码中，生产者线程（producer thread）在条件 **empty** 上等待，给 **fill** 发信号。反过来，消费者线程（consumer thread）在 **fill** 上等待，给 **empty** 发信号。这样一来，上面的第二个问题从设计上就避免了：消费者永远不会意外唤醒消费者，生产者也永远不会意外唤醒生产者。

正确的生产者/消费者方案

我们现在有了一个能工作的生产者/消费者方案，只是还不够通用。最后一处改动是为了提高并发性（concurrency）和效率：我们增加更多的缓冲槽，这样，生产者在睡眠之前可以生产多个值，消费者在睡眠之前也可以消费多个值。只有一个生产者和一个消费者时，这种做法减少了上下文切换（context switch），所以效率更高；有多个生产者或多个消费者

T_{c1}	状态	T_{c2}	状态	T_p	状态	**计数**	说明
c1	运行		就绪		就绪	0	
c2	运行		就绪		就绪	0	
c3	睡眠		就绪		就绪	0	没有数据可取
	睡眠	c1	运行		就绪	0	
	睡眠	c2	运行		就绪	0	
	睡眠	c3	睡眠		就绪	0	没有数据可取
	睡眠		睡眠	p1	运行	0	
	睡眠		睡眠	p2	运行	0	
	睡眠		睡眠	p4	运行	1	缓冲区已满
	就绪		睡眠	p5	运行	1	T_{c1} 被唤醒
	就绪		睡眠	p6	运行	1	
	就绪		睡眠	p1	运行	1	
	就绪		睡眠	p2	运行	1	
	就绪		睡眠	p3	睡眠	1	必须睡眠（已满）
c2	运行		睡眠		睡眠	1	重新检查条件
c4	运行		睡眠		睡眠	0	T_{c1} 取走数据
c5	运行		就绪		睡眠	0	糟糕！唤醒了 T_{c2}
c6	运行		就绪		睡眠	0	
c1	运行		就绪		睡眠	0	
c2	运行		就绪		睡眠	0	
c3	睡眠		就绪		睡眠	0	没有数据可取
	睡眠	c2	运行		睡眠	0	
	睡眠	c3	睡眠		睡眠	0	全都睡着了……

图 30.11：**线程轨迹：有问题的方案（第 2 版）**

（或者两者都有）时，它甚至允许生产或消费并发进行，从而提高并发性。幸运的是，这和当前的方案相比只是一个小改动。

这个正确方案的第一处改动在缓冲区结构本身，以及相应的 `put()` 和 `get()`（图 30.13）。我们还稍微修改了生产者和消费者用来决定是否睡眠的条件。图 30.14 给出了正确的等待和发信号逻辑。只有当所有缓冲槽都已填满时，生产者才会睡眠（p2）；同样，只有当所有缓冲槽都是空的时，消费者才会睡眠（c2）。这样，我们就解决了生产者/消费者问题；可以坐下来，喝杯冰镇啤酒了。

30.3 覆盖条件

我们再来看一个使用条件变量的例子。这个代码案例取自 Lampson 和 Redell 关于 Pilot 的论文 [LR80]，他们也正是最早实现上文所述 **Mesa 语义**（Mesa semantics）的那个团队（他们用的语言叫 Mesa，这个名字就是这么来的）。

他们遇到的问题最好用一个例子来说明：一个简单的多线程内存分配库。图 30.15 给出了一段（简化过的）代码，用来演示这个问题。

```
int loops; // 必须在某处初始化……
cond_t  empty, fill;
mutex_t mutex;

void *producer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        Pthread_mutex_lock(&mutex);
        while (count == 1)
            Pthread_cond_wait(&empty, &mutex);
        put(i);
        Pthread_cond_signal(&fill);
        Pthread_mutex_unlock(&mutex);
    }
}
void *consumer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        Pthread_mutex_lock(&mutex);
        while (count == 0)
            Pthread_cond_wait(&fill, &mutex);
        int tmp = get();
        Pthread_cond_signal(&empty);
        Pthread_mutex_unlock(&mutex);
        printf("%d\n", tmp);
    }
}
```

图 30.12：**生产者/消费者：两个条件变量加 while**

```
int buffer[MAX];
int fill_ptr = 0;
int use_ptr  = 0;
int count    = 0;

void put(int value) {
    buffer[fill_ptr] = value;
    fill_ptr = (fill_ptr + 1) % MAX;
    count++;
}
int get() {
    int tmp = buffer[use_ptr];
    use_ptr = (use_ptr + 1) % MAX;
    count--;
    return tmp;
}
```

图 30.13：**正确的 put 和 get 例程**

```
1  int loops; // 必须在某处初始化……
2  cond_t  empty, fill;
3  mutex_t mutex;
4
5  void *producer(void *arg) {
6      int i;
7      for (i = 0; i < loops; i++) {
8          Pthread_mutex_lock(&mutex);             // p1
9          while (count == MAX)                    // p2
10             Pthread_cond_wait(&empty, &mutex); // p3
11         put(i);                                 // p4
12         Pthread_cond_signal(&fill);             // p5
13         Pthread_mutex_unlock(&mutex);           // p6
14     }
15 }
16
17 void *consumer(void *arg) {
18     int i;
19     for (i = 0; i < loops; i++) {
20         Pthread_mutex_lock(&mutex);             // c1
21         while (count == 0)                      // c2
22             Pthread_cond_wait(&fill, &mutex);  // c3
23         int tmp = get();                        // c4
24         Pthread_cond_signal(&empty);            // c5
25         Pthread_mutex_unlock(&mutex);           // c6
26         printf("%d\n", tmp);
27     }
28 }
```

图 30.14：**正确的生产者/消费者同步**

从代码中可以看到，线程调用内存分配代码时，可能得等一等，直到有更多内存空闲出来。反过来，线程释放内存时，会发信号表示又有内存空闲了。然而，上面的代码有一个问题：应该唤醒哪个等待中的线程？要知道，等待的线程可能不止一个。

设想下面的场景。假设空闲字节数为零；线程 T_a 调用 `allocate(100)`，接着线程 T_b 调用 `allocate(10)`，申请的内存更少。于是 T_a 和 T_b 都在条件上等待，都去睡眠了；空闲字节不够满足其中任何一个请求。

提示：检查条件时用 while（不要用 if）

在多线程程序中检查条件时，用 `while` 循环总是正确的；只用 `if` 语句则不一定正确，取决于发信号的语义。所以，总是用 `while`，你的代码就会按预期运行。

在条件检查外面套上 `while` 循环，还能应对出现**虚假唤醒**（spurious wakeup）的情况。在某些线程库（thread library）中，由于实现上的细节，有可能只发了一次信号，却有两个线程被唤醒 [L11]。虚假唤醒（spurious wakeup）是线程要重新检查所等条件的又一个理由。

 © 2008–25, ARPACI-DUSSEAU

```
1  // 堆上还有多少字节空闲?
2  int bytesLeft = MAX_HEAP_SIZE;
3
4  // 还需要锁和条件变量
5  cond_t  c;
6  mutex_t m;
7
8  void *
9  allocate(int size) {
10     Pthread_mutex_lock(&m);
11     while (bytesLeft < size)
12         Pthread_cond_wait(&c, &m);
13     void *ptr = ...; // 从堆上取内存
14     bytesLeft -= size;
15     Pthread_mutex_unlock(&m);
16     return ptr;
17 }
18
19 void free(void *ptr, int size) {
20     Pthread_mutex_lock(&m);
21     // 把一些内存还给堆
22     bytesLeft += size;
23     Pthread_cond_signal(&c); // 该唤醒谁??
24     Pthread_mutex_unlock(&m);
25 }
```

图 30.15：**覆盖条件：一个例子**

这时，假设第三个线程 T_c 调用 `free(50)`。遗憾的是，它调用 signal 唤醒一个等待线程时，唤醒的可能并不是正确的那个，也就是只等着释放 10 字节的 T_b；而 T_a 应该继续等待，因为空闲内存还不够。因此，图中的代码不能正常工作，因为负责唤醒的线程并不知道该唤醒哪个（或哪些）线程。

Lampson 和 Redell 提出的解决办法很直接：调用 `pthread_cond_broadcast()`，代替上面代码中的 `pthread_cond_signal()`。前者会唤醒所有等待的线程。这样就能保证，所有应该被唤醒的线程都会被唤醒。当然，缺点是可能影响性能，因为我们可能毫无必要地唤醒了许多其他（暂时）还不该醒来的等待线程。这些线程只会醒来，重新检查条件，然后马上回去睡眠。

Lampson 和 Redell 把这样的条件称为**覆盖条件**（covering condition），因为它（保守地）覆盖了所有需要唤醒线程的情况；正如前面讨论的，代价是可能会唤醒太多线程。细心的读者可能还注意到，我们之前也可以用这种办法（参见只用一个条件变量的生产者/消费者问题）。不过在那种情况下，我们有更好的方案可用，所以就用了那个方案。一般来说，如果你发现只有把发信号改成广播（broadcast），程序才能正常工作（而你认为本不应该需要这样），那么你的程序多半有 bug；去把它修好！但在像上面这个内存分配器（memory allocator）这样的情况下，广播可能是现成的最直接的办法。

 © 2008–25, ARPACI-DUSSEAU

30.4 小结

我们介绍了锁之外的又一个重要的同步原语（synchronization primitive）：条件变量。条件变量让线程在程序的某个状态不符合要求时去睡眠，借助它，我们可以干净利落地解决许多重要的同步问题，包括著名的（而且至今仍然重要的）生产者/消费者问题，以及覆盖条件（covering condition）问题。这里本该有一句更有戏剧性的结束语，比如“他热爱老大哥” [O49]。

 © 2008–25, ARPACI-DUSSEAU

参考文献

[D68] "Cooperating sequential processes" by Edsger W. Dijkstra. 1968. Available online here: http://www.cs.utexas.edu/users/EWD/ewd01xx/EWD123.PDF. *Another classic from Dijkstra; reading his early works on concurrency will teach you much of what you need to know.*

[D72] "Information Streams Sharing a Finite Buffer" by E.W. Dijkstra. Information Processing Letters 1: 179–180, 1972. http://www.cs.utexas.edu/users/EWD/ewd03xx/EWD329.PDF *The famous paper that introduced the producer/consumer problem.*

[D01] "My recollections of operating system design" by E.W. Dijkstra. April, 2001. Available: http://www.cs.utexas.edu/users/EWD/ewd13xx/EWD1303.PDF. *A fascinating read for those of you interested in how the pioneers of our field came up with some very basic and fundamental concepts, including ideas like "interrupts" and even "a stack"!*

[H74] "Monitors: An Operating System Structuring Concept" by C.A.R. Hoare. Communications of the ACM, 17:10, pages 549–557, October 1974. *Hoare did a fair amount of theoretical work in concurrency. However, he is still probably most known for his work on Quicksort, the coolest sorting algorithm in the world, at least according to these authors.*

[L11] "Pthread cond signal Man Page" by Mysterious author. March, 2011. Available online: http://linux.die.net/man/3/pthread_cond_signal. *The Linux man page shows a nice simple example of why a thread might get a spurious wakeup, due to race conditions within the signal/wakeup code.*

[LR80] "Experience with Processes and Monitors in Mesa" by B.W. Lampson, D.R. Redell. Communications of the ACM. 23:2, pages 105-117, February 1980. *A classic paper about how to actually implement signaling and condition variables in a real system, leading to the term "Mesa" semantics for what it means to be woken up; the older semantics, developed by Tony Hoare [H74], then became known as "Hoare" semantics, which is a bit unfortunate of a name.*

[O49] "1984" by George Orwell. Secker and Warburg, 1949. *A little heavy-handed, but of course a must read. That said, we kind of gave away the ending by quoting the last sentence. Sorry! And if the government is reading this, let us just say that we think that the government is "double plus good". Hear that, our pals at the NSA?*

 © 2008–25, ARPACI-DUSSEAU

作业（编程）

本作业让你研究一些真实的代码，这些代码用锁和条件变量，实现了本章讨论的生产者/消费者队列的几种形式。你要阅读这些真实的代码，用各种配置运行它们，借此弄清楚哪些做法行得通、哪些行不通，以及其他一些错综复杂的细节。详情请阅读 README。

问题

1. 第一个问题关注 `main-two-cvs-while.c`（能正常工作的方案）。首先，研究一下这段代码。你认为自己清楚运行这个程序时会发生什么吗?
2. 用一个生产者和一个消费者运行，让生产者生产几个值。先用大小为 1 的缓冲区，然后逐渐增大。缓冲区变大以后，代码的行为有什么变化（或者有没有变化）? 把消费者的 sleep string 从默认值（不睡眠）改成 `-C 0,0,0,0,0,0,1` 时，在不同的缓冲区大小（例如 `-m 10`）和不同的生产数量（例如 `-l 100`）下，你预计 `num_full` 会是多少?
3. 如果条件允许，在不同的系统上（例如 Mac 和 Linux）运行这段代码。你在这些系统上看到了不同的行为吗?
4. 我们来看看运行时间。一个生产者、三个消费者、只有一项的共享缓冲区，每个消费者都在 c3 处暂停一秒，你认为下面这次运行要花多长时间?

```
./main-two-cvs-while -p 1 -c 3 -m 1
    -C 0,0,0,1,0,0,0:0,0,0,1,0,0,0:0,0,0,1,0,0,0
    -l 10 -v -t
```

5. 现在把共享缓冲区的大小改成 3（`-m 3`）。这会让总时间有什么不同吗?
6. 现在把睡眠的位置改到 c6（这模拟的是消费者从队列中取出东西，再拿它去做些事情），同样使用只有一项的缓冲区。你预计这种情况下要花多长时间?

```
./main-two-cvs-while -p 1 -c 3 -m 1
    -C 0,0,0,0,0,0,1:0,0,0,0,0,0,1:0,0,0,0,0,0,1
    -l 10 -v -t
```

7. 最后，再把缓冲区大小改成 3（`-m 3`）。现在你预计要花多长时间?
8. 现在来看 `main-one-cv-while.c`。假设只有一个生产者、一个消费者，缓冲区大小为 1，你能设置一个 sleep string，让这段代码出问题吗?
9. 现在把消费者的数量改成两个。你能为生产者和消费者构造出 sleep string，让代码出问题吗?
10. 现在研究 `main-two-cvs-if.c`。你能让这段代码出问题吗? 同样，先考虑只有一个消费者的情况，再考虑有多个消费者的情况。
11. 最后，研究 `main-two-cvs-while-extra-unlock.c`。如果在执行 put 或 get 之前就释放锁，会出现什么问题? 给定 sleep string，你能稳定地让这个问题出现吗? 会发生什么坏事?

 © 2008–25, ARPACI-DUSSEAU